

Quaternions Are Not That Scary

Rotations, Ruby, and a little bit of Hamilton

Ruby Nights Auckland

DragonRuby demo

Why this talk exists

Michael showed a **Three.js** construction-site visualisation.

He mentioned **quaternions** as part of the 3D rotation engine.

The room giggled nervously.

I said:

They are not too scary if viewed the right way.

So now I owe you a talk.

Tonight's promise

Not a full algebra lecture.

Not a derivation of every formula.

Instead:

| Quaternions are a practical language for 3D orientation.

And we will use them from **Ruby**.

The recurring pattern

Mathematics often grows like this:

1. We have a useful number system.
2. We meet an equation or operation it cannot handle.
3. We extend the system.
4. We gain power.
5. We sometimes lose a familiar property.

Natural numbers

We start with:

[
1, 2, 3, 4, \ldots
]

They are excellent for counting.

But the equation

[
 $x + 2 = 1$
]

has no solution in natural numbers.

Integers

Add zero and negatives:

[
\\dots, -2, -1, 0, 1, 2, \\dots
]

Now equations like

[
 $x + a = b$
]

always have an integer solution.

We paid a small conceptual price: numbers can now be “less than nothing”.

Rational numbers

But multiplication creates another problem:

$$\begin{bmatrix} \\ 3x = 2 \\ \end{bmatrix}$$

No integer solution.

So we add fractions:

$$\begin{bmatrix} \\ \frac{2}{3}, \frac{5}{7}, -\frac{11}{4} \\ \end{bmatrix}$$

Now division by non-zero numbers works.

Real numbers

The rationals still have gaps.

The diagonal of a unit square has length:

[
 $\sqrt{2}$
]

It is not rational.

So we complete the number line and obtain the **real numbers**.

Complex numbers

The real numbers still cannot solve:

$$\begin{bmatrix} \\ x^2 = -1 \\ \end{bmatrix}$$

So we add a new number:

$$\begin{bmatrix} \\ i^2 = -1 \\ \end{bmatrix}$$

and obtain:

$$\begin{bmatrix} \\ a + bi \end{bmatrix}$$

What we gained

Complex numbers are not just “numbers with imaginary bits”.

They give us a complete algebraic world:

| Every non-constant polynomial with complex coefficients has a complex root.

From the equation-solving perspective, this is the end of the story.

But geometry continues

Real numbers live on a line.

Complex numbers live on a plane.

And complex numbers of length one rotate that plane:

$$\left[\begin{array}{l} e^{i\theta} = \cos\theta + i\sin\theta \end{array} \right]$$

Multiplication becomes rotation.

A tempting question

If complex numbers rotate the plane so beautifully...

Can we build something similar for **3D space**?

Maybe a number like:

$$\begin{bmatrix} a + bi + cj \end{bmatrix}$$

Hamilton tried.

It did not work.

Hamilton's jump

The breakthrough was not three components.

It was four:

$$\begin{bmatrix} q = w + xi + yj + zk \end{bmatrix}$$

with

$$\begin{bmatrix} i^2 = j^2 = k^2 = ijk = -1 \end{bmatrix}$$

These are the **quaternions**.

The price

Quaternions preserve many familiar rules.

But multiplication is no longer commutative:

[
 $ij = k$
]

while

[
 $ji = -k$
]

The order matters.

This is not a bug

3D rotations themselves do not commute.

Try this with a book:

1. Rotate around the x-axis.
2. Then rotate around the y-axis.

Now reverse the order.

You get a different orientation.

The algebra is telling the truth.

What is a quaternion?

A quaternion has a scalar part and a vector part:

$$\begin{bmatrix} q = (w, \mathbf{v}) \end{bmatrix}$$

or

$$\begin{bmatrix} q = w + xi + yj + zk \end{bmatrix}$$

For rotations, we mostly care about **unit quaternions**.

Axis and angle

A 3D rotation is naturally described by:

- an axis ($\mathbf{u}$)
- an angle (θ)

The corresponding unit quaternion is:

$$\begin{bmatrix} \cos\frac{\theta}{2} \\ \mathbf{u}\sin\frac{\theta}{2} \end{bmatrix}$$

That half-angle is important.

Rotating a point

Treat a 3D point as a “pure” quaternion:

$$\begin{bmatrix} p = 0 + xi + yj + zk \end{bmatrix}$$

Then rotate it by:

$$\begin{bmatrix} p' = qpq^{-1} \end{bmatrix}$$

This is the whole trick.

The Ruby version

```
class Quaternion
  attr_reader :w, :x, :y, :z

  def initialize(w, x, y, z)
    @w, @x, @y, @z = w, x, y, z
  end
end
```

A quaternion is just four numbers.

The magic is in multiplication.

Quaternion multiplication

```
def *(o)
  Quaternion.new(
    w*o.w - x*o.x - y*o.y - z*o.z,
    w*o.x + x*o.w + y*o.z - z*o.y,
    w*o.y - x*o.z + y*o.w + z*o.x,
    w*o.z + x*o.y - y*o.x + z*o.w
  )
end
```

Notice: the order of multiplication matters.

Axis-angle constructor

```
def self.from_axis_angle(axis, angle)
  ax, ay, az = normalize(axis)
  half = angle / 2.0
  s = Math.sin(half)

  Quaternion.new(
    Math.cos(half), ax*s, ay*s, az*s
  )
end
```

One axis. One angle. One object.

Rotating a vector

```
def rotate(v)
  p = Quaternion.new(0, v.x, v.y, v.z)
  r = self * p * conjugate

  Vector3.new(r.x, r.y, r.z)
end
```

This is where algebra touches the screen.

Why DragonRuby?

DragonRuby gives us a tiny Ruby game loop:

```
def tick(args)
  # update state
  # draw frame
end
```

For this demo, we do not need a 3D engine.

We only need:

1. rotate 3D points;
2. project them to 2D;
3. draw lines.

A cube is only data

```
VERTICES = [  
    [-1, -1, -1], [ 1, -1, -1],  
    [ 1,  1, -1], [-1,  1, -1],  
    [-1, -1,  1], [ 1, -1,  1],  
    [ 1,  1,  1], [-1,  1,  1]  
]
```

```
EDGES = [  
    [0, 1], [1, 2], [2, 3], [3, 0],  
    [4, 5], [5, 6], [6, 7], [7, 4],  
    [0, 4], [1, 5], [2, 6], [3, 7]  
]
```


Projection

```
def project(x, y, z)
  camera = 5.0
  scale = camera / (camera - z)

  [
    640 + x * 160 * scale,
    360 + y * 160 * scale
  ]
end
```

Fake 3D can be very convincing.

Drawing the cube

```
points = VERTICES.map do |v|  
  rotated = args.state.orientation.rotate(v)  
  project(*rotated)  
end  
  
EDGES.each do |a, b|  
  args.outputs.lines << line(points[a], points[b])  
end
```

DragonRuby draws ordinary 2D lines.

The quaternion supplies the orientation.

Demo 1: arbitrary axis rotation

```
axis = [1, 1, 1]
angle = Kernel.tick_count * 0.02

args.state.orientation =
    Quaternion.from_axis_angle(axis, angle)
```

The cube rotates around a diagonal axis.

Not x, then y, then z.

One spatial rotation.

Demo 2: order matters

```
if args.inputs.keyboard.key_down.x
  q = Quaternion.from_axis_angle([1,0,0], Math::PI / 2)
  args.state.orientation = q * args.state.orientation
end

if args.inputs.keyboard.key_down.y
  q = Quaternion.from_axis_angle([0,1,0], Math::PI / 2)
  args.state.orientation = q * args.state.orientation
end
```

Press **X then Y**.

Reset.

Press **Y then X**.

What the audience sees

The same two rotations.

The same two angles.

The same cube.

But a different final orientation.

That is:

$$\begin{bmatrix} q_y & q_x \\ q_x & q_y \end{bmatrix}$$

Demo 3: smooth orientation travel

Choose a target orientation:

```
target = Quaternion.from_axis_angle(  
    random_axis,  
    random_angle  
)
```

Then smoothly move towards it:

```
orientation = Quaternion.slerp(  
    start,  
    target,  
    progress  
)
```

This is often why graphics engines love quaternions.

What is SLERP?

SLERP means:

| spherical linear interpolation

Unit quaternions live on a sphere in four dimensions.

SLERP follows a natural curved path between orientations.

On screen, the motion looks smooth and deliberate.

The double-cover surprise

Two quaternions can represent the same rotation:

$$\left[\begin{array}{l} q \quad \text{and} \quad -q \end{array} \right]$$

For rendering, nothing changes.

For interpolation, it matters.

We usually choose the shorter path.

Demo 4: from cube to spectacle

A cube is good for teaching.

But the final object can be more memorable:

- a wireframe ruby gemstone;
- a tiny construction crane;
- a constellation of rotating cubes;
- RUBY letters arranged in 3D;
- a cube with local coordinate axes and trails.

Construction-site callback

A crane gives the talk a nice loop:

Michael began with 3D construction-site visualisation.

We return with a tiny Ruby-made 3D scene.

Possible parts:

- rotating base;
- boom angle;
- hanging hook;
- camera orbit;
- quaternion-controlled orientation.

What DragonRuby is doing

```
def tick(args)
  update_input(args)
  update_orientation(args)
  draw_scene(args)
end
```

That is enough.

The rest is mathematics plus lines on a screen.

The bigger lesson

Each extension gave us something:

[
 $\mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q} \rightarrow \mathbb{R} \rightarrow \mathbb{C} \rightarrow \mathbb{H}$
]

- negatives
- fractions
- continuous lengths
- polynomial roots and plane rotations
- 3D orientation

The price of power

Complex numbers preserve commutativity.

Quaternions do not.

But that loss is not arbitrary.

It matches the world:

- 3D rotations do not commute.

Closing thought

The four components of a quaternion are not four angles.

They are not four dimensions we need to visualise.

Together, they describe **one orientation**.

And with one compact formula:

$$\begin{bmatrix} p' = qpq^{-1} \end{bmatrix}$$

we can rotate a whole 3D world.

Quaternions are strange numbers

But the geometry they describe is the geometry we already live in.